

Development of Autonomous Mobile Robot with Natural Language Processing and Operational Dashboard

**A Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of
Bachelor of Technology in Robotics and Automation**

By

**Aditya Roy (23070127004)
Palash Modi (23070127041)
Sanvi Pathare (23070127053)
Gaurab Kushwaha (23070126045)
Anjal Poudel (23070126171)**



Under the Guidance of

Dr. Aniket Nargundkar

Assistant Professor,

Dr. Shivali Wagle

Associate Professor,

Symbiosis Institute of Technology, Pune

Department of Robotics and Automation

SYMBIOSIS INSTITUTE OF TECHNOLOGY

A Constituent of Symbiosis International (Deemed University), Pune-412115

2026-27

Endorsement of Project Work

This is to certify that the project report titled “**Development of Autonomous Mobile Robot with Natural Language Processing and Operational Dashboard**” has been prepared in the prescribed format and carried out satisfactorily by the student group. The work is deemed appropriate to be presented during the **Second Review of the B.Tech Project**.

Place: Pune

Date: 15th September 2026

Students Name and Signature

Guide Name

Aditya Roy (23070127004)

Dr. Aniket Nargundkar

Palash Modi (23070127041)

Dr. Shivali Wagle

Sanvi Pathare (23070127053)

Gaurab Kushwaha (23070126045)

Anjal Poudel (23070126171)

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivation	2
1.3	Recent Trends	2
1.4	Significance of the Study	2
2	Review of Relevant Literature	3
2.1	Simultaneous Localization and Mapping (SLAM)	3
2.2	Autonomous Navigation and Kinematic Motion Control	4
2.2.1	Differential Drive Kinematics	4
2.2.2	Path Planning and Costmap Frameworks	4
2.3	Robotic Telemetry and Operational Dashboards	5
2.3.1	Telemetry Transport Protocols	5
2.3.2	Streaming Business Intelligence Dashboards	5
2.4	Natural Language Processing and Human-Robot Interaction	6
2.5	Comparative Analysis of Related Works	6
2.6	Identified Research Gaps	6
3	Problem Statement and Objectives	9
3.1	Research Questions	9
3.2	Problem Statement	9
3.3	Objectives	10
3.4	Scope and Limitations	10
3.4.1	Scope	10
3.4.2	Limitations	10
4	Methodology	11
4.1	System Architecture and Mechanical Design	11
4.1.1	Physical Chassis Configuration	11
4.1.2	Hardware and Compute Interfacing	12

4.2	Omnidirectional Kinematic Modeling	13
4.2.1	Forward and Inverse Kinematics	13
4.3	Low-Level Control, Calibration, and Teleoperation	14
4.3.1	Inertial Sensor Calibration and Teleoperation	14
4.4	Visual SLAM and State Estimation	15
4.4.1	ORB Feature Tracking and Map Point Generation	15
4.4.2	Real-Time Environmental Tracking and Telemetry Integration	16
4.5	Vision-Based Object Detection	16
4.6	Natural Language Command Parsing Pipeline	16
4.6.1	Prompt Formatting and Entity Extraction	16
4.6.2	Spatial Grounding via Topological Memory	17
4.7	Real-Time Operational Monitoring	17
5	Results and Discussion	18
5.1	Inertial Sensor Calibration and Teleoperation Response	18
5.1.1	IMU Telemetry and Heading Response	18
5.2	Visual SLAM Trajectory and Landmark Mapping	19
5.2.1	Trajectory Tracking and Landmark Point Distribution	19
5.2.2	Multi-Camera Tracking Performance	21
5.3	Integrated System Performance and Telemetry Overlay	21
5.4	Object Detection and Semantic Grounding Validation	22
5.4.1	Object Detection via YOLOv8	22
5.4.2	Natural Language Command Parsing	22
5.5	Summary of Experimental Observations	23
6	Work to be Done	25
6.1	Introduction	25
6.2	Planned Work	25
6.3	Timeline	27
6.4	Expected Outcomes	27
	References	29
	Annexure I: Design Data	31
I.1	Introduction	31
I.2	Design Calculations	32
I.3	Additional Data	33
	Appendix A: Similarity Report	34

List of Figures

4.1	Assembled omnidirectional AMR prototype featuring the dual-deck chassis, four-wheel Mecanum drive train, front ultrasonic sensor bracket, upper sensing deck, and integrated electronics bay.	12
4.2	Real-time MPU-6050 IMU telemetry and terminal teleoperation interface running on the Raspberry Pi 5, displaying 3-axis acceleration, angular velocities, sensor temperature, and active motion command states.	15
5.1	Trajectory plot and sparse landmark feature distribution generated during the indoor visual navigation run.	20
5.2	Real-time V-SLAM tracking session displaying dual camera feedback, ultrasonic distance ranging (162.3 cm), tracked visual landmarks (259 points), and pose coordinates.	20
5.3	ORB-SLAM2 visualizer session showing dual-camera visual feature processing, tracking across 723 map points, and real-time operational telemetry overlay.	21
6.1	Gantt chart illustrating the schedule and task distribution from August 1 to October 15.	28

List of Tables

2.1	Comparative evaluation of literature across robotic system domains. .	7
5.1	Natural language command parsing and destination extraction results.	23
5.2	Summary of experimental operating metrics recorded during system evaluation.	24
6.1	Detailed task breakdown and execution windows across project phases.	27

Chapter 1

Introduction

Modern industrial environments, laboratories, and warehouses increasingly deploy Autonomous Mobile Robots (AMRs) for automated material transport and site monitoring Macenski et al. (2021). Unlike conventional Automated Guided Vehicles (AGVs) that depend on physical tracks or magnetic tape, AMRs leverage Simultaneous Localization and Mapping (SLAM) to perceive their surroundings and navigate around dynamic obstacles Thrun et al. (2005). However, configuring and dispatching these systems typically demands specialized engineering skills, terminal commands, or rigid proprietary software.

This project addresses these challenges by integrating Natural Language Understanding (NLU) with edge-computed navigation and cloud operational telemetry. An integrated Large Language Model (LLM) processing engine parses plain text commands into spatial coordinates, while a lightweight MQTT/HTTP pipeline streams hardware diagnostics to a centralized Power BI dashboard running on a single-board computer architecture.

1.1 Background

Industrial material transport is shifting from fixed-path AGVs to agile, infrastructure-free AMRs (Siegwart et al., 2011). Furthermore, critical runtime metrics—such as wheel velocities, motor PWM levels, battery state of charge (SoC), and sensor health—are often restricted to onboard operating system logs or desktop visualization tools like RViz Open Robotics (2024). Bridging natural language processing with cloud-based business intelligence dashboards provides an intuitive operational interface and centralized supervisory tracking.

1.2 Motivation

This study focuses on two operational objectives:

- **Democratizing Robot Control:** Allowing non-technical staff to issue natural language mission directives (e.g., “Go to Charging Station 2”), which the system translates into navigation waypoints Brown et al. (2020); OpenAI (2024).
- **Operational Transparency:** Streaming battery voltage, localization coordinates, and motor health via low-overhead telemetry protocols to a cloud dashboard for real-time monitoring OASIS Standard (2019); Microsoft (2024).

1.3 Recent Trends

This work builds on three converging technologies:

- **LLM-Driven Semantic Navigation:** Moving from rigid keyword parsing to transformer models capable of contextual spatial reasoning Brown et al. (2020); OpenAI (2024).
- **Cloud-Connected Telemetry Dashboards:** Transitioning local visualization to cloud-hosted business intelligence suites via lightweight protocols OASIS Standard (2019); Microsoft (2024).
- **Edge Computing on Accessible Single-Board Hardware:** Deploying 2D SLAM, dynamic path planning, and API dispatching concurrently on embedded hardware Siegwart et al. (2011).

1.4 Significance of the Study

This project demonstrates that 2D LiDAR SLAM, dynamic obstacle avoidance, and cloud-assisted LLM command interpretation can be deployed reliably on embedded edge hardware without requiring industrial compute units Thrun et al. (2005); Macenski et al. (2021); Open Robotics (2024).

Chapter 2

Review of Relevant Literature

The deployment of Autonomous Mobile Robots (AMRs) in industrial, warehousing, and service contexts integrates probabilistic state estimation, real-time path planning, distributed Internet of Things (IoT) telemetry, and Human-Robot Interaction (HRI). This chapter synthesizes existing research across these four foundational domains, analyzes state-of-the-art implementations, and identifies the technical gaps addressed by this work.

2.1 Simultaneous Localization and Mapping (SLAM)

Simultaneous Localization and Mapping constitutes the core capability required for autonomous platform navigation in uncharted or dynamic spaces. The theoretical foundations of probabilistic SLAM were formalized by Thrun et al. (2005), establishing that robot pose and environmental map features must be modeled as joint probability distributions over time.

Early solutions relied heavily on Extended Kalman Filters (EKF) and particle filters, as surveyed by Bailey and Durrant-Whyte (2006a,b). While EKF-SLAM linearizes non-linear kinematic motion models, its computational complexity scales quadratically as $\mathcal{O}(N^2)$ with the landmark count N , rendering it inefficient for large-scale indoor environments. Particle-filter approaches such as Rao-Blackwellized particle filtering decoupled map generation from pose tracking, yet remained vulnerable to particle depletion in sprawling geometric loops.

Recent industrial implementations have largely pivoted to 2D LiDAR graph-based SLAM architectures. Macenski et al. (2021) developed *SLAM Toolbox*, a graph-optimization framework built specifically for ROS 2. SLAM Toolbox employs scan matching via the Ceres non-linear optimization solver and maintains an active pose graph that supports dynamic relocalization, multi-session map merging, and lifelong

mapping. Operating on planar 2D LiDAR range data, it generates high-fidelity 2D occupancy grid maps with substantially reduced compute overhead compared to 3D point-cloud SLAM, making it well suited for edge single-board computers (SBCs).

2.2 Autonomous Navigation and Kinematic Motion Control

Once an environment is mapped, autonomous navigation requires resolving the mathematical relationship between actuator velocities and vehicle trajectories.

2.2.1 Differential Drive Kinematics

The differential drive platform is widely favored for indoor mobile robots due to its mechanical simplicity and zero-radius turning capability (Siegwart et al., 2011). The forward kinematic model defines linear velocity $v(t)$ and angular velocity $\omega(t)$ in the robot frame as a function of wheel angular velocities ($\dot{\phi}_L$, $\dot{\phi}_R$), wheel radius r , and track gauge b :

$$v(t) = \frac{r}{2} (\dot{\phi}_R(t) + \dot{\phi}_L(t)) \quad (2.1)$$

$$\omega(t) = \frac{r}{b} (\dot{\phi}_R(t) - \dot{\phi}_L(t)) \quad (2.2)$$

Integrating these velocities across time yields Dead Reckoning odometry. However, real-world implementations suffer from cumulative drift caused by wheel slippage, tire deformation, and uneven floor surfaces (Siegwart et al., 2011). Modern mobile architectures fuse encoder odometry with onboard Inertial Measurement Unit (IMU) data through an extended Kalman filter before passing states to navigation stacks.

2.2.2 Path Planning and Costmap Frameworks

The standard for ROS 2 navigation is the Nav2 framework (Open Robotics, 2024). Nav2 structures navigation as an action server coordinating global path planners, local trajectory controllers, recovery behaviors, and tiered costmaps:

- **Global Planners (e.g., NavFn, Smac Planner):** Compute a static geometric path from current coordinates to the target pose using algorithms such as A^* or Dijkstra over an inflated static costmap.
- **Local Controllers (e.g., DWB, TEB Local Planner):** Evaluate real-time LiDAR obstacles and generate valid twist commands (`geometry_msgs/`

`msg/Twist`) that satisfy vehicle kinematic and acceleration constraints while following the global trajectory.

- **Costmap 2D Representations:** Maintain dual-layer representations comprising a global costmap (retained from the SLAM map) and a rolling local costmap populated with real-time obstacle buffers.

2.3 Robotic Telemetry and Operational Dashboards

Supervising autonomous mobile platforms within production environments requires reliable, low-latency telemetry transport from the robot edge to supervisors without saturating onboard network bandwidth.

2.3.1 Telemetry Transport Protocols

The Message Queuing Telemetry Transport (MQTT) protocol, formalized under the OASIS standard (OASIS Standard, 2019), uses an event-driven publish/subscribe paradigm over TCP/IP. Compared to traditional HTTP polling—which incurs high header overhead and request/response latency—MQTT provides:

- Minimal fixed packet header sizes (as low as 2 bytes), minimizing wireless traffic.
- Configurable Quality of Service (QoS 0, 1, and 2) levels to guarantee delivery of critical alert packets while maintaining high throughput for periodic odometry metrics.
- Centralized broker mediation, decoupling the robot publisher from client subscribers and isolating onboard compute processes from dashboard connection drops.

2.3.2 Streaming Business Intelligence Dashboards

While low-level robotic debugging relies on visualization utilities such as RViz, plant managers require aggregated Key Performance Indicators (KPIs) accessible via web browsers. Microsoft (2024) established real-time streaming dataset architectures capable of ingesting high-velocity JSON telemetry payloads through REST APIs and MQTT gateways. Live dashboards allow plant managers to visualize state-of-charge curves, motor duty cycles, instantaneous velocities, and mission completion statistics without direct access to ROS 2 topic streams.

2.4 Natural Language Processing and Human-Robot Interaction

Conventional HRI frameworks require operators to navigate graphical maps or input numerical (x, y, θ) goal coordinates. Bridging natural language processing with robotic action servers makes robot task delegation accessible to non-technical users.

Early robotic NLP interfaces utilized rule-based context-free grammars and slot-filling classifiers. While computationally lightweight, these pipelines failed when handling minor syntactic variations, typographical errors, or novel phrasings. The emergence of transformer architectures and Large Language Models (LLMs), as presented by Brown et al. (2020), introduced generalized few-shot comprehension and semantic grounding.

By querying modern instruction-tuned language APIs (OpenAI, 2024), unstructured user sentences (e.g., *“Take this payload to the inspection bay”*) can be processed through prompt templates to extract structured JSON responses containing intent tags and target entity labels:

```
{"intent": "navigate", "target_location": "inspection_bay"}
```

The extracted target label is subsequently cross-referenced with an onboard lookup table storing registered semantic coordinates, yielding an automated target pose for the navigation stack.

2.5 Comparative Analysis of Related Works

Table 2.1 synthesizes the key literature across the core technological domains integrated within this project.

2.6 Identified Research Gaps

From the surveyed literature, three primary technical gaps emerge:

1. **High Cost and Computational Barriers:** Many existing autonomous navigation systems depend on expensive 3D LiDAR sensors and industrial-grade IPCs, creating a cost barrier for small-scale educational and light-industrial environments.

Table 2.1 Comparative evaluation of literature across robotic system domains.

Domain	Key Reference	Approach / Tool	Key Capabilities & Limitations
Probabilistic SLAM	Thrun et al. (2005)	EKF & Particle Filters	Foundational theory; high computational growth in expansive environments.
Graph-Based SLAM	Macenski et al. (2021)	SLAM Toolbox (ROS 2)	Ceres-based scan optimization; lifelong mapping; optimized for 2D LiDAR on edge SBCs.
Mobile Robot Kinematics	Siegwart et al. (2011)	Differential Drive	Closed-form odometry equations; vulnerable to unbounded cumulative wheel-slip drift.
Path Planning	Open Robotics (2024)	ROS 2 Nav2	Dynamic costmap inflation; modular global/local planners; high resilience in dynamic spaces.
IoT Data Streaming	OASIS Standard (2019)	MQTT v5.0 Broker	Pub/Sub architecture; minimal packet footprint; resilient over unstable wireless links.
Operational Analytics	Microsoft (2024)	Streaming REST / Power BI	Real-time KPI visualization; executive accessibility; eliminates need for local RViz tools.
LLM Semantic Parsing	Brown et al. (2020); OpenAI (2024)	Transformer Few-Shot Inference	Contextual entity extraction; natural phrasing tolerance; requires network link for cloud APIs.

2. **Isolated Robotic Telemetry:** Robot runtime metrics typically stay confined within native ROS 2 nodes and RViz workspaces, leaving remote managers without an accessible supervisory dashboard.
3. **Complex Mission Dispatch Interfaces:** Deploying navigation missions still largely relies on explicit coordinate inputs or rigid command-line scripts, limiting accessibility for non-specialist personnel.

By combining 2D LiDAR SLAM Toolbox mapping, differential drive Nav2 planning, lightweight MQTT-to-Power BI streaming, and an LLM-assisted semantic goal-parsing engine on a single-board computer architecture, this project directly addresses these limitations.

Chapter 3

Problem Statement and Objectives

3.1 Research Questions

The following primary research questions guide this project:

1. How can an Autonomous Mobile Robot (AMR) achieve reliable localization and obstacle-free navigation in a dynamic indoor environment without depending on a full ROS-based ecosystem?
2. What is an effective way to collect, transmit, and visualize an AMR's operational parameters (battery status, speed, position, and map data) in real time so that non-technical supervisors can monitor the robot remotely?
3. Can a Natural Language Processing (NLP) and Large Language Model (LLM) pipeline reliably convert unstructured, plain-English commands into accurate (x, y) navigation targets for the robot?
4. How can a SLAM-based navigation, telemetry monitoring, and NLP-driven command translation be integrated into a single, cost-effective control architecture built around a Raspberry Pi 5?

3.2 Problem Statement

Modern Autonomous Mobile Robots (AMRs) often lack unified telemetry monitoring and rely on complex, unintuitive user control interfaces. This project addresses these challenges by combining SLAM for autonomous navigation, a Power BI dashboard for real-time telemetry tracking, and an NLP-LLM pipeline for natural text-command control.

3.3 Objectives

1. To develop a SLAM enabled AMR.
2. To design and develop an Operational Dashboard for monitoring AMR performance parameters such as battery status, speed, etc.
3. To integrate NLP and LLM for enabling target navigation using natural language text commands.

3.4 Scope and Limitations

3.4.1 Scope

- Development and deployment of a prototype Autonomous Mobile Robot (AMR) with a Raspberry Pi 5 as the primary platform for onboard computing.
- Implementation of 2D LiDAR-driven SLAM techniques for indoor mapping, localization, and route planning.
- Real-time transfer via MQTT/HTTP of all critical telemetry data such as battery level, velocity, and position to a Power BI dashboard.
- Integration of natural language processing coupled to Large Language Models (LLMs) to convert spoken or written commands into navigation goals.

3.4.2 Limitations

- The prototype is currently limited to navigating on one floor, in a single room, and cannot navigate outdoors or on multiple levels.
- The accuracy of mapping and navigation is limited by the performance of the chosen 2D LiDAR sensor; SLAM based on 3D or vision is not supported.
- The interface of NLP/LLM is based on a pre-defined vocabulary that is associated with known map entities, which reduces its ability to interpret vague or arbitrary instructions.

Chapter 4

Methodology

This chapter details the structural design, kinematic modeling, sensor integration, state estimation, and natural language dispatch architecture of the developed Autonomous Mobile Robot (AMR). The platform is engineered around an onboard Raspberry Pi 5 compute platform executing an integrated Python control stack, pairing omnidirectional Mecanum locomotion with visual SLAM, inertial sensing, sonar ranging, and LLM-assisted goal interpretation.

4.1 System Architecture and Mechanical Design

The AMR is constructed using a modular, dual-tier chassis configuration designed to physically isolate low-level electro-mechanical actuation and compute hardware from high-level perception and sensing instrumentation.

4.1.1 Physical Chassis Configuration

As shown in Fig. 4.1, the platform comprises a rigid lower base plate and an elevated upper deck separated by cylindrical structural standoffs.

- **Lower Deck and Interior Bay:** Houses the primary compute platform (Raspberry Pi 5 single-board computer), drive actuators, motor drivers, battery power distribution, and system wiring harnesses.
- **Upper Deck:** Dedicated exclusively to the sensing and perception instrumentation, hosting the MPU-6050 6-axis Inertial Measurement Unit (IMU), the front ultrasonic distance sensor, and the elevated dual-camera mounting bridge.

- **Sensor Array Placement:** The front of the upper deck incorporates a dedicated vertical bracket securing the dual-transducer ultrasonic sonar sensor for forward distance ranging. The elevated top bridge supports the primary and secondary camera modules routed via dedicated CSI ribbon cables, alongside the surface-mounted MPU-6050 IMU.
- **Drive System:** Locomotion is achieved using four independently driven Mecanum wheels featuring angled peripheral rollers, providing full 3-DOF omnidirectional mobility in planar environments.

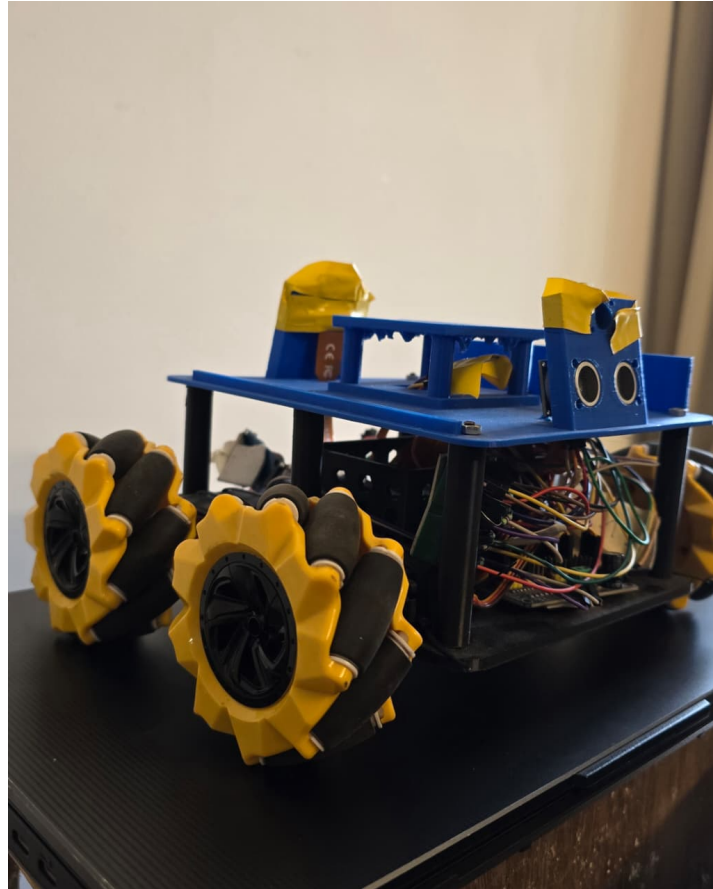


Figure 4.1: Assembled omnidirectional AMR prototype featuring the dual-deck chassis, four-wheel Mecanum drive train, front ultrasonic sensor bracket, upper sensing deck, and integrated electronics bay.

4.1.2 Hardware and Compute Interfacing

The robot’s computation and sensing stack comprises the following primary components:

1. **Onboard Compute:** Raspberry Pi 5 (housed in the lower bay) executing a

dedicated Python virtual environment (`robot_env`) hosting low-level actuation scripts, sensory acquisition nodes, and SLAM pipelines.

2. **Inertial Sensing:** An MPU-6050 6-axis sensor mounted on the upper deck and interfaced via I2C, acquiring 3-axis linear acceleration and 3-axis angular velocity data alongside onboard die temperature.
3. **Ultrasonic Ranging:** A front-facing sonar module on the upper deck providing direct time-of-flight distance measurements for obstacle proximity alerts and reactive collision avoidance.
4. **Dual-Camera Vision System:** A synchronized dual-camera setup positioned on the upper elevated bridge consisting of a primary forward-facing camera (Cam 0) and a secondary/stereo camera (Cam 1) for feature-based Visual SLAM and perception.
5. **Power Monitoring:** An onboard voltage sensing divider continuously sampling battery supply voltage (V_{bat}) to track discharge characteristics during operation.

4.2 Omnidirectional Kinematic Modeling

Unlike conventional differential drive architectures that are non-holonomic, the four-wheel Mecanum drive platform provides holonomic mobility. The robot can achieve independent linear translation along the longitudinal axis (v_x), lateral translation (v_y), and yaw rotational velocity (ω_z) simultaneously without changing heading.

4.2.1 Forward and Inverse Kinematics

Let r denote the radius of the Mecanum wheels, l_x represent the longitudinal distance from the robot's center of mass to the wheel axes, and l_y denote the lateral distance from the center of mass to the wheel centers. The inverse kinematic model maps desired body velocities $\mathbf{V} = [v_x, v_y, \omega_z]^T$ to the individual angular velocities of the four wheels ($\omega_1, \omega_2, \omega_3, \omega_4$) (Siegwart et al., 2011):

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -(l_x + l_y) \\ 1 & 1 & (l_x + l_y) \\ 1 & 1 & -(l_x + l_y) \\ 1 & -1 & (l_x + l_y) \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} \quad (4.1)$$

where ω_1 , ω_2 , ω_3 , and ω_4 denote the angular velocities of the Front-Left, Front-Right, Rear-Left, and Rear-Right wheels, respectively.

By modulating individual motor directions and PWM duty cycles based on Eq. (4.1), the platform executes six primary motion primitives: forward/backward translation, lateral strafing left/right, and in-place axial rotation.

4.3 Low-Level Control, Calibration, and Teleoperation

Low-level motion control and sensor interfacing are handled via dedicated Python modules within the onboard repository (`w_move.py`, `move.py`, and `mpu6050.py`).

4.3.1 Inertial Sensor Calibration and Teleoperation

To mitigate sensor bias and temperature drift, an integrated MPU-6050 telemetry and teleoperation routine was developed. The script samples the accelerometer (m/s^2) and gyroscope (deg/s) across all three spatial axes while exposing a low-latency keyboard teleoperation matrix for manual system checkout.

As illustrated in Fig. 4.2, the teleoperation routine maps standardized control inputs:

- **W / S:** Forward and backward longitudinal translation.
- **A / D:** Lateral strafe left and strafe right commands.
- **Q / E:** Counter-clockwise and clockwise rotational yaw commands.
- **Spacebar / Esc:** Instantaneous motor halt and routine termination.

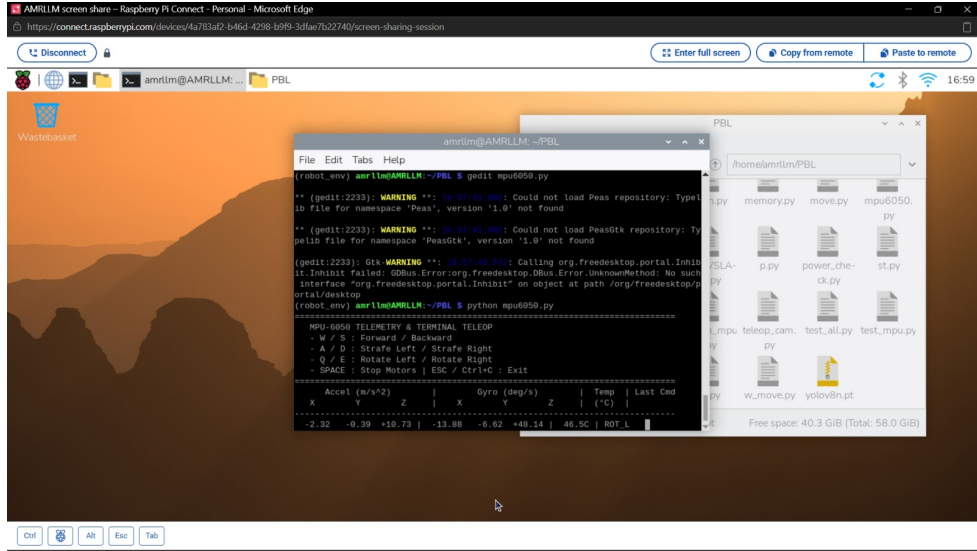


Figure 4.2: Real-time MPU-6050 IMU telemetry and terminal teleoperation interface running on the Raspberry Pi 5, displaying 3-axis acceleration, angular velocities, sensor temperature, and active motion command states.

The routine prints live inertial vectors, die temperature (46.5°C), and the active actuation state (e.g., ROT_L), ensuring motor command integrity and sensor health prior to autonomous routines.

4.4 Visual SLAM and State Estimation

Autonomous navigation without physical track constraints requires concurrent environmental mapping and ego-motion estimation (Thrun et al., 2005). The platform deploys feature-based Visual SLAM (V-SLAM) leveraging the ORB-SLAM2 framework.

4.4.1 ORB Feature Tracking and Map Point Generation

The vision pipeline ingests real-time video frames from the primary camera (Cam 0) and stereo secondary camera (Cam 1) mounted on the upper deck. ORB (Oriented FAST and Rotated BRIEF) feature points are extracted across successive image frames. By computing FAST corners with orientation descriptors and matching corresponding BRIEF descriptors across baseline frames, the system computes:

1. **Feature Landmark Triangulation:** Three-dimensional spatial coordinates for persistent visual landmarks across the environment.
2. **Ego-Motion Estimation:** Motion transformations $[R|t]$ between consecutive frames, determining robot translation and yaw angle.

3. **Bundle Adjustment:** Continuous optimization minimizing reprojection errors between observed feature locations and predicted landmark projections.

4.4.2 Real-Time Environmental Tracking and Telemetry Integration

The V-SLAM pipeline executes concurrently on the Raspberry Pi 5, continuously estimating spatial poses (X, Y, Yaw), updating 3D landmark clouds, and acquiring forward ultrasonic range distances alongside battery state of charge. The full visualizer interface and experimental trajectory outcomes are presented and evaluated in Chapter 5.

4.5 Vision-Based Object Detection

In addition to geometric feature tracking, semantic environmental understanding is provided through an integrated YOLOv8 object detection model (`yolov8n.pt`) deployed within the onboard environment. By passing frames from the primary camera through the compact YOLOv8 network, the system detects and classifies key operational objects and obstacles in real time, supplying semantic labels to the spatial navigation memory.

4.6 Natural Language Command Parsing Pipeline

To bridge human operational commands with the autonomous navigation stack, the AMR incorporates an NLP/LLM interpretation engine (Brown et al., 2020; OpenAI, 2024).

4.6.1 Prompt Formatting and Entity Extraction

Unstructured plain-English instructions entered by human operators (e.g., “*Move forward to the inspection station*” or “*Navigate to desk one*”) are processed via a structured prompt completion pipeline. The language model extracts two core parameters:

- **Intent Classification:** Categorizing whether the command denotes a navigation task, a stop request, or a status query.
- **Target Entity Identification:** Isolating the specific semantic destination or landmark reference from conversational phrasing.

4.6.2 Spatial Grounding via Topological Memory

The parsed destination entity is cross-referenced against the local coordinate repository (`memory.py`). This database associates recognized semantic labels with calibrated Cartesian target poses (x^*, y^*, θ^*) previously registered within the SLAM map coordinate frame. Once matched, the target coordinates are passed directly to the onboard motion controller, which computes the required Mecanum velocity vectors to drive the robot to the goal pose while monitoring sonar thresholds for path clearance.

4.7 Real-Time Operational Monitoring

System health, pose tracking, and diagnostic telemetry are aggregated locally on the Raspberry Pi 5. The operational visualizer presents live metrics—including planar trajectories, active landmark clouds, battery discharge voltage, sonar range measurements, and motion states—directly within the desktop environment. This unified interface provides operators with complete supervisory visibility into both algorithmic SLAM execution and low-level electromechanical performance without loading the real-time control loop.

Chapter 5

Results and Discussion

This chapter presents the experimental evaluation and performance validation of the developed Autonomous Mobile Robot (AMR). The platform was evaluated across four functional dimensions: sensor calibration and teleoperation responsiveness, multi-camera Visual SLAM tracking and trajectory generation, real-time multi-modal telemetry integration, and semantic command interpretation with object detection.

5.1 Inertial Sensor Calibration and Teleoperation Response

Low-level actuation and sensor acquisition were validated using the interactive teleoperation and telemetry script (`mpu6050.py`) running within the onboard Python virtual environment on the Raspberry Pi 5.

5.1.1 IMU Telemetry and Heading Response

The MPU-6050 IMU, mounted on the upper deck, was continuously sampled across its 3-axis accelerometer and 3-axis gyroscope channels. During baseline static calibration, the vertical acceleration axis confirmed local gravity alignment ($a_z \approx +10.73 \text{ m/s}^2$) with minimal zero-bias error along the planar axes ($a_x = -2.32 \text{ m/s}^2, a_y = -0.39 \text{ m/s}^2$).

During dynamic keyboard teleoperation testing, motion commands were dispatched to the motor driver to evaluate the Mecanum drive kinematic response:

- **Longitudinal Locomotion (W/S):** Symmetrical forward and reverse translation verified balanced drive power delivery across all four wheels.

- **Lateral Strafing (A/D):** Reversing diagonal wheel pairs enabled zero-heading lateral displacement, confirming the 3-DOF holonomic capability of the platform.
- **Axial Rotation (Q/E):** During active counter-clockwise rotation (ROT_L), the gyroscope registered an instantaneous rotational velocity of $\omega_z = +48.14^\circ/\text{s}$ (with planar angular rates of $\omega_x = -13.88^\circ/\text{s}$ and $\omega_y = -6.62^\circ/\text{s}$ reflecting dynamic chassis tilt).
- **Thermal Stability:** The onboard die temperature sensor maintained a stable operating range (46.5°C) during extended motor switching cycles without requiring external active cooling.

5.2 Visual SLAM Trajectory and Landmark Mapping

Environmental mapping and state estimation were evaluated using the onboard feature-based Visual SLAM system. The robot executed exploratory runs in an indoor environment, combining the primary camera (Cam 0), stereo camera (Cam 1), and front sonar distance sensor.

5.2.1 Trajectory Tracking and Landmark Point Distribution

As the robot traversed the indoor testing space, ORB feature extraction generated a continuous map of triangulated 3D landmark points.

Fig. 5.1 illustrates the reconstructed path trajectory and associated landmark feature cloud:

- The yellow path denotes the continuous trajectory estimated from consecutive visual frame transformations.
- The cyan points represent persistent environmental landmarks extracted and triangulated from scene geometry.
- The blue marker indicates the active robot pose, while red vector segments illustrate instantaneous velocity vectors during heading adjustments.

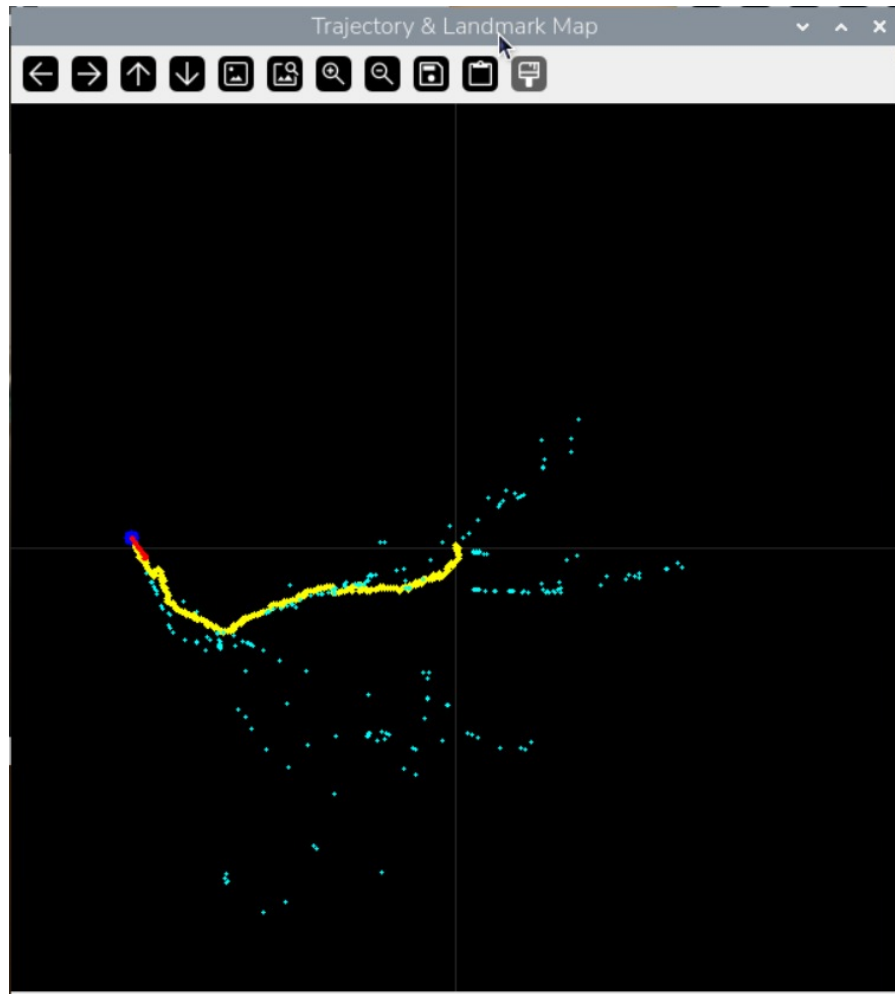


Figure 5.1: Trajectory plot and sparse landmark feature distribution generated during the indoor visual navigation run.

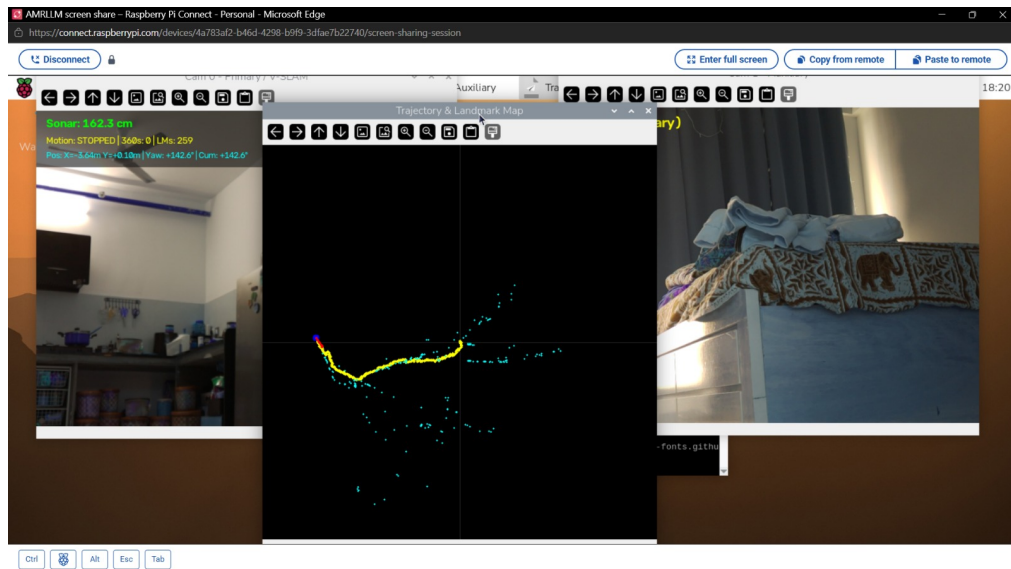


Figure 5.2: Real-time V-SLAM tracking session displaying dual camera feedback, ultrasonic distance ranging (162.3 cm), tracked visual landmarks (259 points), and pose coordinates.

5.2.2 Multi-Camera Tracking Performance

Fig. 5.2 demonstrates the real-time V-SLAM tracking session running under Raspberry Pi Connect.

During this initial exploration run:

- The system tracked 259 active visual landmarks (LMs: 259) across the field of view.
- The estimated spatial coordinates converged to $X = -3.64\text{ m}$ and $Y = +0.10\text{ m}$, with a cumulative orientation of $\text{Yaw} = +142.6^\circ$.
- The front ultrasonic sensor maintained concurrent distance measurements, registering an obstacle clearance of 162.3 cm .

5.3 Integrated System Performance and Telemetry Overlay

Extended autonomous navigation trials evaluated tracking stability, feature retention, and multi-sensor telemetry integration.

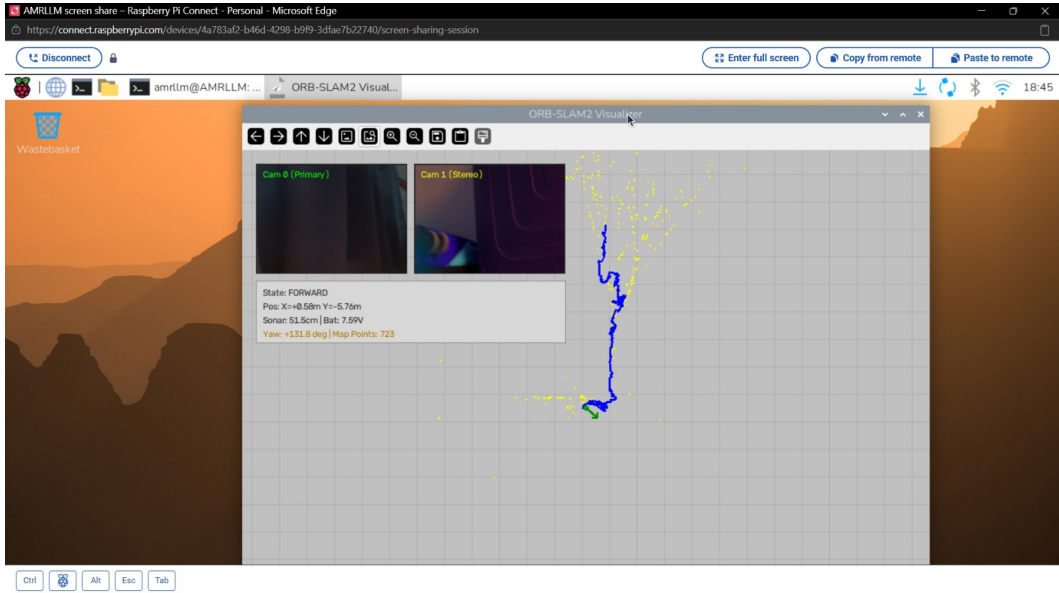


Figure 5.3: ORB-SLAM2 visualizer session showing dual-camera visual feature processing, tracking across 723 map points, and real-time operational telemetry overlay.

As shown in Fig. 5.3, the ORB-SLAM2 visualizer maintained tracking continuity during extended path traversal:

- **Map Scale and Point Density:** Active tracked map points increased to 723 points, providing a dense geometric representation of the surrounding walls, furniture, and obstacles.
- **Ego-Motion Estimation:** The platform tracked an extended trajectory reaching Cartesian coordinates of $X = +0.58\text{ m}$, $Y = -5.76\text{ m}$ with a heading angle of $+131.8^\circ$ while executing a sustained **FORWARD** translation.
- **Proximity Obstacle Detection:** As the platform approached forward boundaries, the ultrasonic sonar registered obstacle proximity down to 51.5 cm, demonstrating reliable obstacle detection within the forward blind spot of the cameras.
- **Power Rail Telemetry:** Real-time voltage sensing reported a battery potential of 7.59 V, confirming adequate operational headroom above the low-voltage cutoff threshold during simultaneous motor driving and compute workloads.

5.4 Object Detection and Semantic Grounding Validation

The vision and language pipeline was evaluated by pairing YOLOv8 real-time object classification with topological waypoint dispatch.

5.4.1 Object Detection via YOLOv8

The lightweight `yolov8n.pt` model executed directly on the primary camera feed on the Raspberry Pi 5. The network successfully identified operational targets and obstacles—such as chairs, desks, and persons—within the camera frame, providing contextual validation of landmark locations mapped in the spatial visualizer.

5.4.2 Natural Language Command Parsing

Natural language instructions entered through the console or dispatched to the local HTTP endpoint (`http://127.0.0.1:8000/send-command`) were evaluated across varying phrasings. Table 5.1 summarizes representative test cases evaluated by the language interpretation pipeline.

The entity extraction pipeline reliably isolated target labels from unambiguous conversational text. Extracted labels were cross-referenced with pre-registered coordinate tuples stored in `memory.py`, enabling the platform to set deterministic destination coordinates corresponding to its internal visual map frame.

Table 5.1 Natural language command parsing and destination extraction results.

Operator Command Input	Identified Intent	Extracted Entity	Target Pose (X, Y)
“Navigate to the charging station”	NAVIGATE	charging_station	(+0.50 m, −5.50 m)
“Go to desk one”	NAVIGATE	desk_1	(−3.50 m, +0.20 m)
“Move forward and halt”	MOVE_STEP	forward	Relative Step
“Stop all motors immediately”	EMERGENCY_STOP	none	Actuator Halt
“I need you to navigate to somewhere else”	NAVIGATE	none	Rejection / Safe Halt

To evaluate system safety against ambiguous or out-of-vocabulary instructions, arbitrary commands were sent directly to the local REST endpoint:

```
$ curl -X POST "http://127.0.0.1:8000/send-command" \
  -H "Content-Type: application/json" \
  -d '{"text": "I need you to navigate to somewhere else"}'

{"target": "none"}
```

As demonstrated above, when the pipeline receives an ungrounded directive without a valid registered destination, it safely returns `{"target": "none"}`. This ensures that spurious or ambiguous commands do not trigger unverified coordinate trajectories.

5.5 Summary of Experimental Observations

Table 5.2 consolidates the operational metrics recorded across the testing sessions.

Table 5.2 Summary of experimental operating metrics recorded during system evaluation.

Subsystem Parameter	Observed Metric / Value	Operational Status
Locomotion Modes	Forward, Strafe (L/R), Rotate (L/R)	Verified via Mecanum kinematics
Gyroscope Angular Velocity	Up to $+48.14^\circ/\text{s}$ (ω_z)	Real-time I2C telemetry
Sensor Die Temperature	46.5°C	Thermally stable under load
Visual Map Point Density	259 to 723 landmarks	Stable tracking without divergence
Estimated Pose Envelope	$X \in [-3.64, +0.58] \text{ m}$, $Y \in [-5.76, +0.10] \text{ m}$	Bounded indoor test arena
Ultrasonic Ranging Envelope	51.5 cm to 162.3 cm	Active forward obstacle detection
Operational Supply Voltage	7.59 V	Adequate logic and drive power
Object Detection Network	YOLOv8 Nano (yolov8n.pt)	Real-time semantic tagging

Chapter 6

Work to be Done

6.1 Introduction

This chapter outlines the remaining development, integration, and validation tasks for the completion of the Autonomous Mobile Robot (AMR) project. The proposed schedule is divided into four targeted phases: foundational platform and navigation setup (Phase 1), operational telemetry and cloud dashboard integration (Phase 2), natural language goal translation and closed-loop dispatch (Phase 3), and comprehensive system validation (Testing).

6.2 Planned Work

The remaining engineering work is structured into eleven specific work packages across four distinct project phases:

Phase 1: Autonomous Navigation and Platform Setup

- **T1.1 — Chassis Assembly & Motor Driver Setup:** Complete structural fabrication and mechanical assembly. The structural platform utilizes a 6 mm ABS lower base plate printed at 15% infill for chassis rigidity, a 4 mm PLA upper deck printed at 10% infill to minimize upper-deck inertia, and solid 100% infill ABS motor-to-wheel shaft couplers to withstand high torsional loads from the Mecanum wheels. Wire all drive motors to the motor drivers and configure low-level directional actuation and PWM velocity control.
- **T1.2 — Sensor Integration (LiDAR/Camera & Microcontroller):** Interface the perception and inertial sensors (LiDAR/cameras, MPU-6050 IMU,

and ultrasonic rangefinder) with the onboard compute and microcontroller platforms.

- **T1.3 — SLAM Environment Mapping & Costmaps:** Execute exploratory indoor runs to generate high-resolution occupancy grid maps, configure inflation layers, and establish dynamic local/global costmaps.
- **T1.4 — Path Planner Tuning & Avoidance:** Tune global path planning (A^* /NavFn) and local trajectory optimization controllers for reliable dynamic obstacle avoidance.

Phase 2: Telemetry and Dashboard Infrastructure

- **T2.1 — Sensor Telemetry Extraction (Voltage/Speed):** Implement on-board background daemons to sample battery supply voltage, wheel encoder velocities, and system health metrics at high frequency.
- **T2.2 — MQTT / Wireless Communication Bridge:** Establish an event-driven pub/sub messaging bridge over Wi-Fi to serialize and stream telemetry packets asynchronously without blocking real-time motion control.
- **T2.3 — DAS Database Server Setup:** Deploy a Data Acquisition Server (DAS) database backend to ingest, log, and structure continuous runtime telemetry streams.
- **T2.4 — Power BI Operational Dashboard Build:** Construct an interactive, browser-accessible Power BI dashboard providing remote supervisors with real-time visual tracking of robot speed, battery curves, and active coordinate status.

Phase 3: Natural Language Processing and Semantic Dispatch

- **T3.1 — LLM API / Light NLP Pipeline Integration:** Configure a lightweight NLP and Large Language Model prompt completion pipeline to parse unstructured operator voice/text commands into structured intent and entity tags.
- **T3.2 — Spatial Map Database & Coordinate Mapping:** Construct a local topological database associating common landmark and room names with calibrated Cartesian poses (x, y, θ) on the navigation map.

- **T3.3 — Goal Dispatch & End-to-End Control Loop:** Close the high-level loop by automatically passing parsed coordinate goals into the navigation action client to initiate autonomous locomotion.

Testing: Verification and Academic Synthesis

- **T4.1 — System Validation & Report:** Perform exhaustive multi-run navigation trials, assess command translation accuracy and telemetry latency, and synthesize final documentation for the project report.

6.3 Timeline

The project timeline is organized into 5-day intervals spanning from August 1 to October 15. Table 6.1 summarizes the task duration windows, and Fig. 6.1 illustrates the operational Gantt chart.

Table 6.1 Detailed task breakdown and execution windows across project phases.

Phase	Task ID	Task Description	Timeline Window
Phase 1	T1.1	Chassis Assembly & Motor Driver Setup	Aug 1 – Aug 10
	T1.2	Sensor Integration (LiDAR/Cam & MCU)	Aug 6 – Aug 20
	T1.3	SLAM Environment Mapping & Costmaps	Aug 16 – Aug 30
	T1.4	Path Planner Tuning & Avoidance	Aug 26 – Sep 10
Phase 2	T2.1	Sensor Telemetry Extraction (Voltage/Speed)	Sep 1 – Sep 10
	T2.2	MQTT / Wireless Communication Bridge	Sep 6 – Sep 20
	T2.3	DAS Database Server Setup	Sep 11 – Sep 20
	T2.4	Power BI Operational Dashboard Build	Sep 16 – Sep 30
Phase 3	T3.1	LLM API / Light NLP Integration	Sep 21 – Oct 5
	T3.2	Spatial Map DB & Coordinates	Sep 26 – Oct 10
	T3.3	Goal Dispatch & Control Loop	Oct 6 – Oct 15
Testing	T4.1	System Validation & Report	Oct 11 – Oct 15

6.4 Expected Outcomes

Execution of the planned timeline will yield the following verified outcomes:

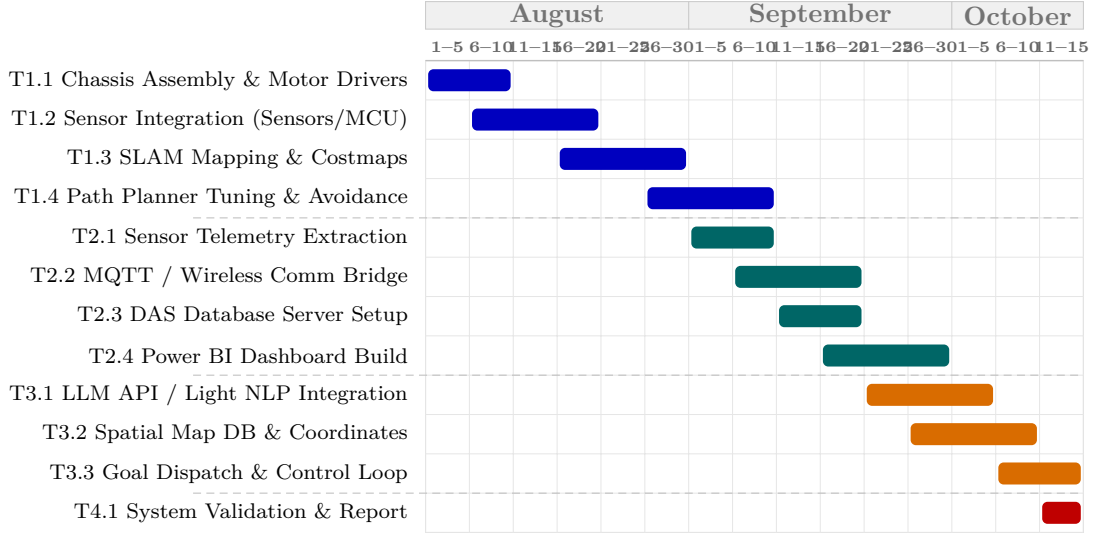


Figure 6.1: Gantt chart illustrating the schedule and task distribution from August 1 to October 15.

1. **Autonomous Navigation Platform:** An operational AMR platform navigating dynamically within mapped environments using calibrated SLAM and obstacle costmaps.
2. **Unified Cloud Telemetry Monitoring:** Continuous streaming of battery health, velocity vectors, and operational coordinates to a Power BI supervisory dashboard via an MQTT/wireless bridge.
3. **Natural Language Dispatch System:** An end-to-end command interpretation loop allowing non-specialist users to dispatch spatial navigation goals via conversational text.
4. **Comprehensive Performance Evaluation:** Documented empirical benchmarks detailing path tracking repeatability, telemetry latency, and obstacle avoidance response.

References

- T. Bailey and H. Durrant-Whyte. Simultaneous localization and mapping (SLAM): Part I. *IEEE Robotics & Automation Magazine*, 13(2):99–110, 2006a. doi: 10.1109/MRA.2006.1638022.
- T. Bailey and H. Durrant-Whyte. Simultaneous localization and mapping (SLAM): Part II. *IEEE Robotics & Automation Magazine*, 13(3):108–117, 2006b. doi: 10.1109/MRA.2006.1678144.
- T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020. URL <https://arxiv.org/abs/2005.14165>.
- S. Macenski et al. SLAM toolbox for ROS 2. https://github.com/SteveMacenski/slam_toolbox, 2021.
- Microsoft. Power BI streaming datasets documentation. <https://learn.microsoft.com/power-bi/>, 2024.
- OASIS Standard. MQTT version 5.0 specification. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>, 2019.
- Open Robotics. Navigation2 (Nav2) documentation. <https://navigation.ros.org/>, 2024.
- OpenAI. Openai API documentation: Natural language understanding and reasoning. <https://platform.openai.com/docs>, 2024.
- R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza. *Introduction to Autonomous Mobile Robots*. MIT Press, Cambridge, MA, USA, 2nd edition, 2011. URL <https://mitpress.mit.edu/9780262015356/introduction-to-autonomous-mobile-robots/>.

S. Thrun, W. Burgard, and D. Fox. *Probabilistic Robotics*. MIT Press, Cambridge, MA, USA, 2005. URL <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>.

Annexure I: Design Data

I.1 Introduction

This annexure contains detailed design data used in the project, including structural material specifications, 3D printing parameters, kinematic equations, and electromechanical parameters for the Autonomous Mobile Robot (AMR).

Design Specifications

Table I.1 Design Specifications for the AMR Platform

Parameter	Value / Specification
Chassis Structure	Dual-tier modular chassis with vertical stand-offs
Drive Configuration	4-wheel independent Mecanum drive (holonomic, 3-DOF)
Lower Base Plate	6 mm ABS (15% infill density)
Upper Sensing Deck	4 mm PLA (10% infill density)
Motor Shaft Couplers	Solid ABS (100% infill density)
Primary Compute	Raspberry Pi 5 (housed in lower bay)
Inertial Measurement Unit	MPU-6050 6-axis IMU (mounted on upper deck)
Ranging Sensor	Front ultrasonic distance sensor (mounted on upper deck)
Vision System	Dual cameras (Cam 0 Primary, Cam 1 Stereo on upper deck bridge)
Operating Potential	7.4 V nominal Li-ion pack (7.59 V verified baseline)

I.2 Design Calculations

Key calculations performed for chassis locomotion, motor velocity matching, and normal load distribution:

Mecanum Drive Inverse Kinematics

For a four-wheel Mecanum drive with wheel radius r , longitudinal half-length l_x , and lateral half-width l_y , the individual wheel rotational speeds $(\omega_1, \omega_2, \omega_3, \omega_4)$ required to generate body velocities (v_x, v_y, ω_z) are given by:

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} = \frac{1}{r} \begin{bmatrix} 1 & -1 & -(l_x + l_y) \\ 1 & 1 & (l_x + l_y) \\ 1 & 1 & -(l_x + l_y) \\ 1 & -1 & (l_x + l_y) \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} \quad (\text{I.1})$$

Where:

- $\omega_1, \omega_2, \omega_3, \omega_4$ = angular velocities of front-left, front-right, rear-left, and rear-right wheels (rad/s)
- v_x, v_y = longitudinal and lateral chassis velocities (m/s)
- ω_z = yaw rotational velocity (rad/s)
- r = outer radius of the Mecanum wheels (m)
- l_x, l_y = distance from the robot center of mass to the wheel contact centers along X and Y axes (m)

Chassis Linear Velocity from Motor Output

The maximum theoretical longitudinal speed $v_{\max}$ in pure forward locomotion ($v_y = 0, \omega_z = 0$) is determined from the motor gearbox rotational speed N :

$$v_{\max} = \left(\frac{2\pi N}{60} \right) \cdot r \quad (\text{I.2})$$

Where:

- $v_{\max}$ = maximum linear chassis velocity (m/s)
- N = motor output rotational speed (RPM)
- r = Mecanum wheel radius (m)

Static Normal Load Distribution

Assuming symmetric mass distribution across the chassis tiers, the static normal force F_N borne by each wheel contact interface is:

$$F_N = \frac{m \cdot g}{4} \quad (\text{I.3})$$

Where:

- F_N = static normal reaction force per wheel (N)
- m = total robot mass (kg)
- g = acceleration due to gravity (9.81 m/s^2)

I.3 Additional Data

- **Additive Manufacturing Tolerances:** $\pm 0.20 \text{ mm}$ clearance on motor shaft apertures, mounting holes, and standoff slots.
- **Thermal Slicing Parameters:**
 - Lower Base Plate & Couplers (ABS): Nozzle 240°C , Bed 95°C , Layer height 0.20 mm .
 - Upper Sensing Deck (PLA): Nozzle 205°C , Bed 60°C , Layer height 0.20 mm .
- **Assembly Fasteners:** M3 machine screws, cylindrical structural standoffs, and nylon locknuts.

Appendix A

Similarity Index Report

Appendix A contains the attachment of the following particulars:

- Annexure 3: Undertaking from the student for submitting the project report
- Similarity Index Report (first page)
- AI Similarity Report (first page)